



CC1002 - Introducción a la Programación

CLASE 5: RECURSIVIDAD

Departamento de Ciencias de la Computación (DCC) - FCFM
UNIVERSIDAD DE CHILE



Nadia Decar (nadiadecar@ug.uchile.cl)

Eduardo Godoy (egodoy@dcc.uchile.cl)

Drawing hands

Litografía

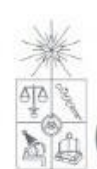
Impresa en 1984

Maurits Cornelis Escher

Artista holandés

Paradoja

Una no existe sin la otra



Recursión en la naturaleza

Un árbol es una rama

Una rama es una rama de donde salen dos ramas

Una rama es una rama de donde salen dos ramas

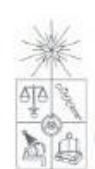
Una rama es una rama de donde salen dos ramas

Una rama es una rama de donde salen dos ramas

Una rama es una rama de donde salen dos ramas

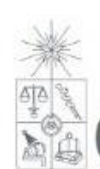
...

En algún momento de la rama sale una hoja



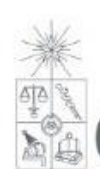
Recursión y humor

- En un glosario:
 - Recursión: *vea Recursión*
- Una explicación rápida:
 - Para entender la recursión, usted debe entender la recursión
- Acrónimos recursivos:
 - PHP: “PHP Hypertext Preprocessor”
 - GNU: “GNU's not Unix”
- Personal (reglas de mi casa)
 - Regla 1: Mamá siempre tiene la razón
 - Regla 2: Lea la regla 1



Recursión

- Un objeto/función es recursivo/a si:
 1. Tiene un caso (o casos) base que no requiere recursión
 2. Un conjunto de reglas que reducen los otros casos hacia el caso base.
- Computación:
 - Se implementa con funciones
 - Hay una condición que separa el caso base



Recursión

- Ejemplo en matemática:

- $2^0 = 1$
- $2^n = 2 * 2^{n-1}$

Caso Base: $2^0 = 1$

Otros casos: $2^N = 2 * 2^{N-1}$

- Calcular 2^3

$$2^3 = 2 * 2^2$$

- Calcular 2^3

$$2^3 = 2 * 2^2$$

$$2^3 = 2 * (2 * 2^1)$$

- Calcular 2^3

$$2^3 = 2 * 2^2$$

$$2^3 = 2 * (2 * 2^1)$$

$$2^3 = 2 * (2 * (2 * 2^0))$$

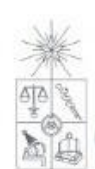
- Calcular 2^3

$$2^3 = 2 * 2^2$$

$$2^3 = 2 * (2 * 2^1)$$

$$2^3 = 2 * (2 * (2 * 2^0))$$

$$2^3 = 2 * (2 * (2 * 1))$$



Recursión

- Ejemplo en matemática:

- $2^0 = 1$
- $2^n = 2 * 2^{n-1}$

- Calcular 2^3

$$2^3 = 2 * 2^2$$

$$2^3 = 2 * (2 * 2^1)$$

$$2^3 = 2 * (2 * (2 * 2^0))$$

$$2^3 = 2 * (2 * (2 * 1))$$

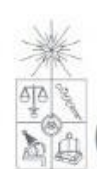
$$2^3 = 2 * (2 * (2))$$

$$2^3 = 2 * (4)$$

$$2^3 = 8$$

Caso Base: $2^0 = 1$

Otros casos: $2^N = 2 * 2^{N-1}$



Recursión

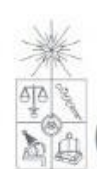
```
# potencia2: int -> int
# dado un exponente, calcula 2 elevado al exponente
# positivo
# Ejemplo: potencia2(3) retorna 8
# Ejemplo: potencia2(0) retorna 1
def potencia2(exponente):
    assert exponente >= 0
    if exponente == 0:
        return 1
    else:
        return 2 * potencia2(exponente - 1)
# Test
assert potencia2(3) == 8
assert potencia2(0) == 1
```

Caso base

Si el exponente es 0, la función retorna 1 sin volver a llamarse.

Caso recursivo

Multiplika 2 por potencia2(exponente - 1), acercándose al caso base.

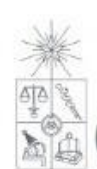


Recursión

```
# potencia2: int -> int
# dado un exponente, calcula 2 elevado al exponente
# Ejemplo: potencia2(3) retorna 8
# Ejemplo: potencia2(0) retorna 1
def potencia2(exponente) :
    if exponente == 0 :
        return 1
    else :
        return 2 * potencia2(exponente - 1)
# Test
assert potencia2(3) == 8
assert potencia2(0) == 1
```

Caso Base: $2^0 = 1$

Otros casos: $2^N = 2 * 2^{N-1}$



3

```
def potencia2(exponente) :  
    if exponente == 0 :  
        return 1  
    else :  
        return 2 * potencia2( exponente -1)
```

2

```
def potencia2(exponente):  
    if exponente == 0 :  
        return 1  
    else :  
        return 2 * potencia2( exponente -1)
```

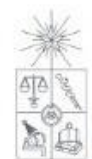
1

1

```
def potencia2(exponente):  
    if exponente == 0 :  
        return 1  
    else :  
        return 2 * potencia2( exponente -1)
```

0

```
def potencia2(exponente):  
    if exponente == 0 :  
        return 1  
    else :  
        return 2 * potencia2( exponente -1)
```



```
def potencia2(exponente) :  
    if exponente == 0 :  
        return 1  
    else :  
        return 2 * potencia2( exponente -1)
```

```
def potencia2(exponente) :  
    if exponente == 0 :  
        return 1  
2    else :  
        return 2 * potencia2( exponente -  
1)
```

```
def potencia2(exponente) :  
    if exponente == 0 :  
        return 1  
    else :  
        return 2 * potencia2( exponente -  
1)
```

```
def potencia2(exponente) :  
    if exponente == 0 :  
        return 1  
    else :  
        return 2 * potencia2( exponente -  
1)
```

8

4

1

2

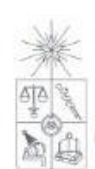


Recursión: X^n

- Construya la función potencia la cual retorna el resultado de elevar un número dado a un exponente dado (X^n)
 - El exponente es un número entero, mayor o igual a cero
 - Recuerde que 0^0 no está definido

def **potencia**(numero, exponente) :

: :



Recursión

```
# potencia: num int -> num
# dado un número y un exponente, calcula
# el número elevado al exponente
# Ejemplo: potencia(2, 3) retorna 8
# Ejemplo: potencia(5.0, 2) retorna 25.0
# Ejemplo: potencia(0, 0) genera un error
def potencia(numero, exponente):
    # Se valida la precondition
    assert not (numero == 0 and exponente == 0)

    if exponente == 0:
        return 1
    else:
        return numero * potencia(numero, exponente - 1)

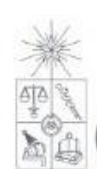
# Test
assert potencia(2, 3) == 8
assert potencia(5.0, 2) == 25.0
```

Precondición

El assert descarta potencia(0, 0), que no está definida.

Recursión

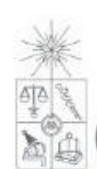
Si el exponente es 0 retorna 1; si no, multiplica el número por potencia(numero, exponente - 1).



Recursión

```
# potencia: num int -> num
# dado un número y un exponente, calcula
# el número elevado al exponente
# Ejemplo: potencia(2, 3) retorna 8
# Ejemplo: potencia(5.0, 2) retorna 25.0
# Ejemplo: potencia(0, 0) genera un error
def potencia(numero, exponente) :
    # Se valida la precondición
    assert not (numero == 0 and exponente == 0)

    if exponente == 0 :
        return 1
    else :
        return numero * potencia( numero, exponente - 1)
# Test
assert potencia(2, 3) == 8
assert potencia(5.0, 2) == 25.0
```



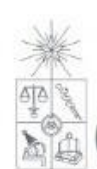
Recursión: total de dígitos

- Construya la función `digitos(numero)`, la cual retorna el número de dígitos de un número entero:
 - `digitos(3425)` → retorna 4
 - `digitos(23)` → retorna 2
 - `digitos(9)` → retorna 1
- Indicación:
Recuerde que
$$3425 // 10 \rightarrow 342$$
$$4 // 10 \rightarrow 0$$



Descubriendo la recursión

- Cuántos dígitos hay en los números del 0 al 9 $\rightarrow$ 1
 - `numero // 10 == 0`

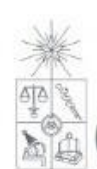


Descubriendo la recursión

8	7	2	3	1	9	4
---	---	---	---	---	---	---

- Cuántos dígitos hay en los números del 0 al 9
→ 1
 - `numero // 10 == 0`
- tome el número y "achíquelo"
 - dividir por 10

8	7	2	3	1	9
---	---	---	---	---	---



Descubriendo la recursión

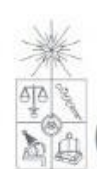
- Cuántos dígitos hay en los números del 0 al 9 $\rightarrow 1$
 - `numero // 10 == 0`

- tome el número y "achíquelo"
 - dividir por 10

8	7	2	3	1	9	4
---	---	---	---	---	---	---

8	7	2	3	1	9
---	---	---	---	---	---

- `digitos([8 7 2 3 1 9 4]) = 1 + digitos([8 7 2 3 1 9])`



Descubriendo la recursión

- Cuántos dígitos hay en los números del 0 al 9 $\rightarrow$ 1

- `numero // 10 == 0`

- tome el número y "achíquelo"

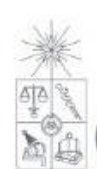
8	7	2	3	1	9	4
---	---	---	---	---	---	---

- dividir por 10

8	7	2	3	1	9
---	---	---	---	---	---

- `digitos([8 7 2 3 1 9 4]) = 1 + digitos([8 7 2 3 1 9])`

- `digitos([8 7 2 3 1 9 4]) = 1 + digitos([8 7 2 3 1 9 4] // 10)`



Recursión: total de dígitos

```
# digitos: int -> int
# dado un número retorna el número de
# digitos que tiene ese número
# Ejemplo: digitos(234) retorna 3
# Ejemplo: digitos(4) retorna 1
# Ejemplo: digitos(23421) retorna 5
def digitos(numero):
    if numero // 10 == 0:
        return 1
    else:
        return 1 + digitos(numero // 10)

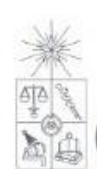
# Test
assert digito(234) == 3
assert digito(4) == 1
```

Caso base

Un número menor que 10 tiene un solo dígito: retorna 1.

Caso recursivo

Cuenta 1 dígito y vuelve a llamarse con `numero // 10`, quitando el último dígito.



Recursión: suma de dígitos

Ejercicio

Construya la función **sumarDigitos()**, que retorna la suma de los dígitos de un número entero.

Indicación

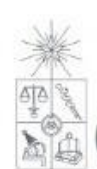
- La división entera quita el último dígito.
- El módulo entrega el último dígito.

Ejemplos

```
sumarDigitos(3425) → 14  
sumarDigitos(23) → 5  
sumarDigitos(9) → 9
```

Recuerde

```
3425 // 10 → 342  
4 % 10 → 4
```



Descubriendo la recursión

- Cuántos dígitos hay en los números del 0 al 9 → el número
 - `numero // 10 == 0`

- tome el número y "achíquelo"
 - dividir por 10

8	7	2	3	1	9	4
---	---	---	---	---	---	---

8	7	2	3	1	9
---	---	---	---	---	---

- `sumaDigitos`(

8	7	2	3	1	9	4
---	---	---	---	---	---	---

) = 4 + `sumaDigitos`(

8	7	2	3	1	9
---	---	---	---	---	---

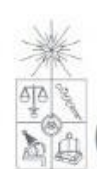
)
- `sumaDigitos`(

8	7	2	3	1	9	4
---	---	---	---	---	---	---

) = 4 + `sumaDigitos`(

8	7	2	3	1	9	4
---	---	---	---	---	---	---

 // 10)



Recursión: suma de dígitos

```
# sumaDigitos: int -> int
# dado un número retorna la suma de los
# dígitos que tiene ese número
# Ejemplo: sumaDigitos(234) retorna 9
# Ejemplo: sumaDigitos(4) retorna 4
# Ejemplo: sumaDigitos(23421) retorna 12
def sumarDigitos(numero):
    if numero // 10 == 0:
        return numero
    else:
        return (numero % 10) + sumarDigitos(numero // 10)

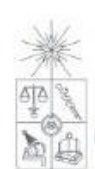
# Test
assert sumaDigitos(234) == 9
assert sumaDigitos(4) == 4
```

Caso base

Si el número tiene un solo dígito, la suma es el número mismo.

Caso recursivo

Suma el último dígito ($\text{numero} \% 10$) y sigue con el resto ($\text{numero} // 10$).



Recursión: Divisible por 3

Ejercicio

Un número es divisible por 3 si la suma recursiva de sus dígitos es 3, 6 o 9.

Construya la función **esDivisibleX3(numero)**, que retorna True o False.

Indicación

- No puede usar / , // ni %
- Puede llamar a sumarDigitos(...)
- Con receta de diseño y 2 tests

Ejemplos

$27 \rightarrow 2+7 = 9$

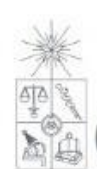
$214578324 \rightarrow 36 \rightarrow 3+6 = 9$

Casos de prueba

`esDivisibleX3(9) → True`

`esDivisibleX3(23) → False`

`esDivisibleX3(3429) → True`



Conejos cariñosos

Mes 0 Una pareja de conejos inmaduros se escapa.

Mes 1 La pareja ya es adulta.

Mes 2 Nace su primer par de crías: macho y hembra.

Mes 3 Los padres tienen otro par de crías y la primera camada ya es adulta.

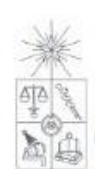
Mes 4 Los padres y la primera camada tienen crías; la segunda camada madura.

Mes 5 Se suman las crías de los padres, de los hijos y de la segunda camada.

... y así sucesivamente cada mes.

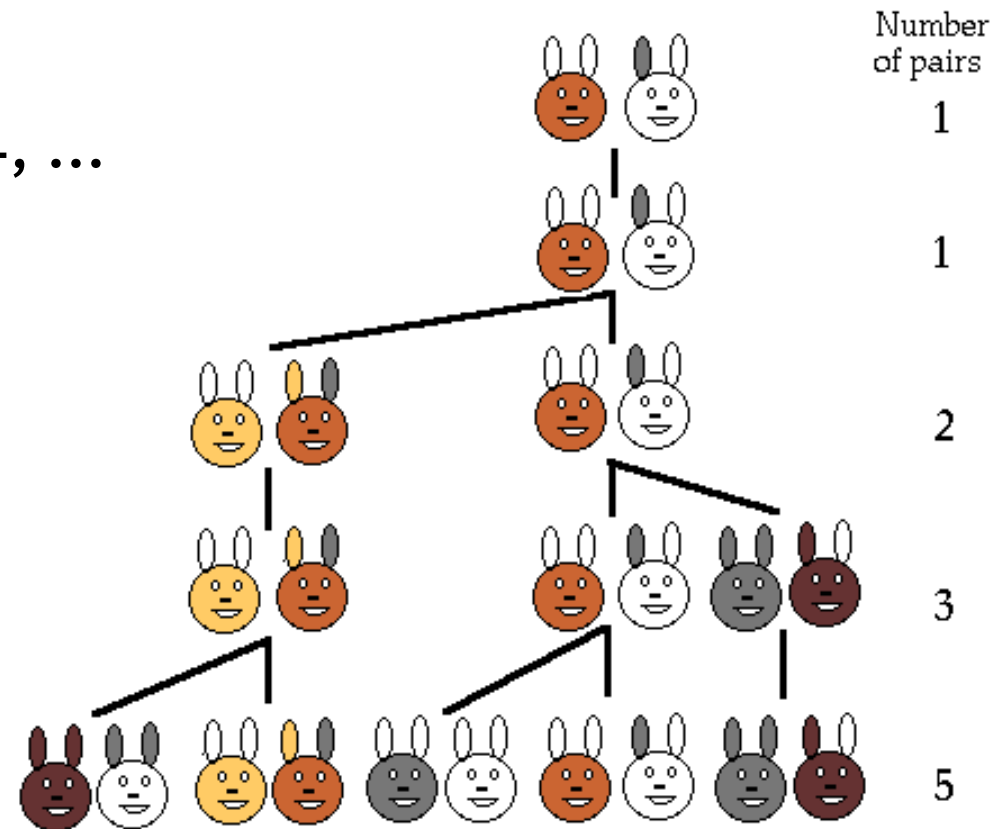


¿Cuántas parejas de conejos hay en la etapa n ?



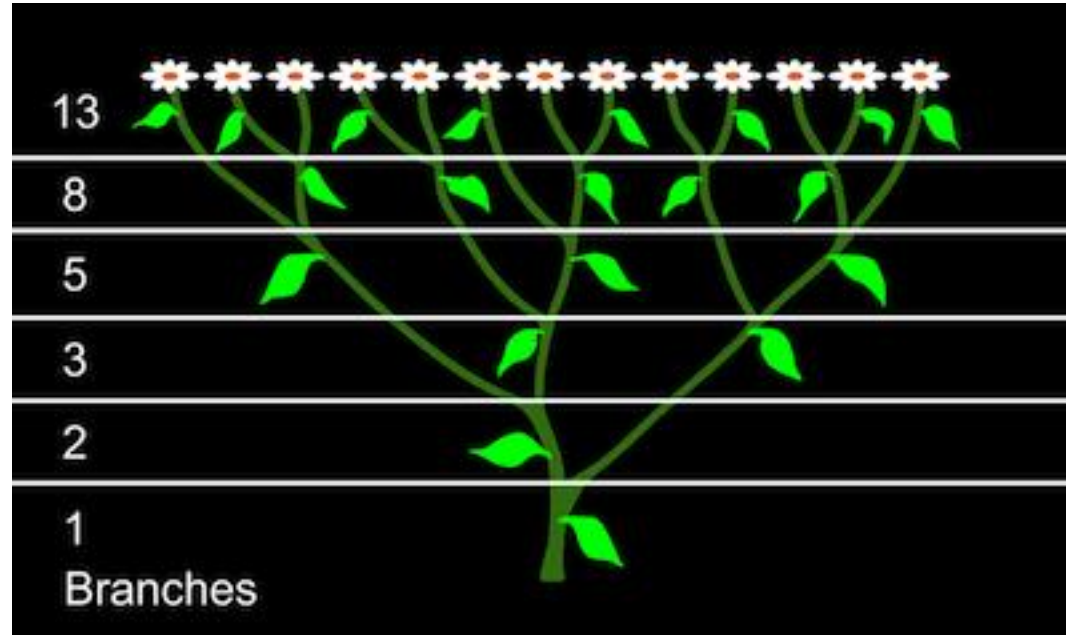
Conejos cariñosos

- Las parejas crecen en:
1, 1, 2, 3, 5, 8, 13, 21, 34, ...
- ¿Qué tan extraños son estos números?

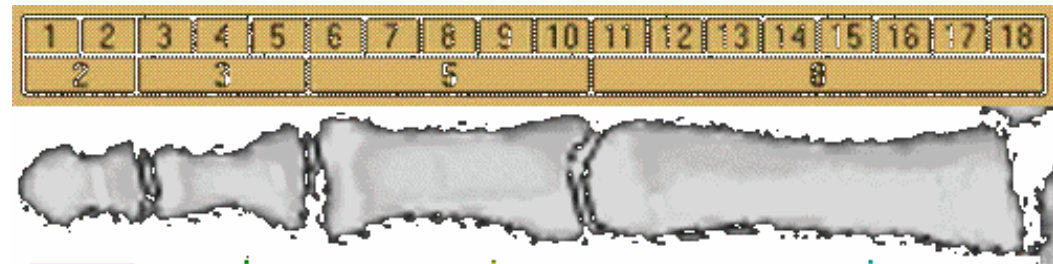


Naturaleza

- Las ramas de una planta



- ¡Su dedo!



Fibonacci

La serie se llama Fibonacci

Regla

$\text{Fibonacci}(0) \rightarrow 1$

$\text{Fibonacci}(1) \rightarrow 1$

$\text{Fibonacci}(n) \rightarrow \text{Fibonacci}(n-1)$
 $+ \text{Fibonacci}(n-2)$

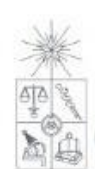
Cada término es la suma de los dos anteriores.

Leonardo de Pisa

Matemático italiano del siglo XIII, conocido como Fibonacci, quien describió la serie.

Y entonces...

¿Cuántas parejas de conejos hay al mes n ?



Fibonacci (Versión 1.0)

```
# fibonacci: int -> int
# dado un número determina el valor de
# la serie de fibonacci para ese número
# Ejemplo: fibonacci(1) retorna 1
# Ejemplo: fibonacci(7) retorna 13
def fibonacci(n):
    if n == 0:
        return 1
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

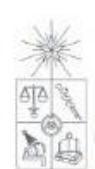
# Test
assert fibonacci(1) == 1
assert fibonacci(7) == 13
```

Dos casos base

fibonacci(0) y fibonacci(1) retornan 1 sin llamarse de nuevo.

Caso recursivo

Suma los dos términos anteriores: fibonacci(n-1) + fibonacci(n-2).



Fibonacci (Versión 2.0)

```
# fibonacci: int -> int
# dado un número determina el valor de
# la serie de fibonacci para ese número
# Ejemplo: fibonacci(1) retorna 1
# Ejemplo: fibonacci(7) retorna 13
def fibonacci(n):
    if n == 0 or n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

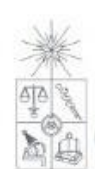
# Test
assert fibonacci(1) == 1
assert fibonacci(7) == 13
```

Un solo caso base

Los dos casos base se unen en una condición: $n == 0$ or $n == 1$.

Caso recursivo

Igual que antes: $\text{fibonacci}(n-1) + \text{fibonacci}(n-2)$.



Series

- Determine la suma de $1 + 2 + 3 + \dots + n$

- ¿Qué valor tiene la serie:

- Indicación

- Use la función potencia2()

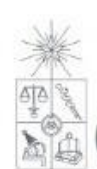
$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \dots + \frac{1}{2^n}$$

- ¿Qué valor tiene la serie:

- Indicación

- Notar que es negativo si el número es par

$$1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} - \frac{1}{6} + \dots - \frac{1}{n}$$



Suma1n = 1 + 2 + 3 + ... + n

```
# suma1n: int -> int
# dado un número n, determina el valor de
# la serie 1+2+3+ ... + n
# Ejemplo: suma1n(1) retorna 1
# Ejemplo: suma1n(5) retorna 15
def suma1n(n):
    if n == 1:
        return 1
    else:
        return n + suma1n(n - 1)

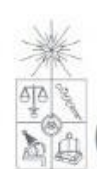
# Test
assert suma1n(1) == 1
assert suma1n(5) == 15
```

Caso base

Si n es 1, la suma es 1 y la recursión se detiene.

Caso recursivo

Suma n y sigue con la serie hasta n - 1: n + suma1n(n-1).



Series

- Determine la suma de $1 + 2 + 3 + \dots + n$

- ¿Qué valor tiene la serie:

- Indicación

- Use la función potencia2()

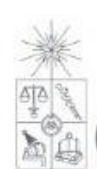
$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \dots + \frac{1}{2^n}$$

- ¿Qué valor tiene la serie:

- Indicación

- Notar que es negativo si el número es par

$$1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} - \frac{1}{6} + \dots - \frac{1}{n}$$



Unodiv2n =

$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \dots + \frac{1}{2^n}$$

```
# unodiv2n: int -> num
# dado un número n, determina el valor de
# la serie 1+1/2+1/4+1/8+ ... +1/(2^n)
# Ejemplo: unodiv2n(0) retorna 1
# Ejemplo: unodiv2n(5) retorna 1.96875
def unodiv2n(n):
    if n == 0:
        return 1
    else:
        return (1.0 / potencia2(n)) + unodiv2n(n - 1)

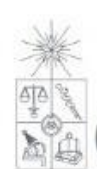
# Test
assert unodiv2n(0) == 1
assert unodiv2n(5) == 1.96875
```

Caso base

Si n es 0, la serie vale 1 y la recursión termina.

Caso recursivo

Suma el término $1/2^n$ y sigue con la serie hasta n - 1.



Series

- Determine la suma de $1 + 2 + 3 + \dots + n$

- ¿Qué valor tiene la serie:

- Indicación

- Use la función potencia2()

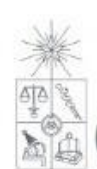
$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \dots + \frac{1}{2^n}$$

- ¿Qué valor tiene la serie:

- Indicación

- Notar que es negativo si el número es par

$$1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} - \frac{1}{6} + \dots - \frac{1}{n}$$



Masmenos1divn

```
# masmenos1divn: int -> num
# dado un número n, determina el valor de
# la serie 1-1/2+1/3-1/4+ ... 1/n
# Ejemplo: masmenos1divn(1) retorna 1
# Ejemplo: masmenos1divn(5) retorna 0.783333
def masmenos1divn(n):
    if n == 1:
        return 1
    else:
        if espar(n):
            return (-1.0/n) + masmenos1divn(n - 1)
        else:
            return (1.0/n) + masmenos1divn(n - 1)

# Test
assert masmenos1divn(1) == 1
assert masmenos1divn(2) == 0.5
```

$$1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} - \frac{1}{6} + \dots \frac{1}{n}$$

Caso base

Si n es 1, la serie vale 1.

Signo alternado

espar(n) decide si el término suma o resta.

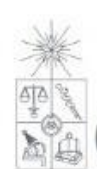


Ejercicio propuesto

- Programe la función $\cos(x, n)$ con la serie:

$$\cos(x, n) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots + \frac{(-1)^n x^{2n}}{(2n)!}$$

- Use recursividad
- Use la función **factorial(n)** del apunte
- Implemente un diálogo simple para que el auxiliar pueda probar su ejercicio.



Ejercicio propuesto

Una serie armónica es aquella que suma los inversos multiplicativos de los enteros positivos.

(El inverso multiplicativo de un número es aquel que, al multiplicarse por el número original, da como resultado 1)

Inverso multiplicativo de 2 es $\frac{1}{2}$

Inverso multiplicativo de $\frac{2}{3}$ es $\frac{3}{2}$

$$H(n) = \begin{cases} 1 & \text{si } n = 1 \\ \frac{1}{n} + H(n-1) & \text{si } n > 1 \end{cases}$$

Se pide programar la función Harmonica

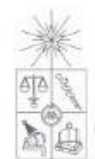
(debe incluir receta)

Turtle

- Idea original del lenguaje Logo
 - Desarrollado en 1996 por Wally Feurzig y Seymour Papert
- Este “tortuga” parte ubicada en (0, 0)
 - Centro de la pantalla
- Puede caminar (forward / backward)
 - Si avanza dibuja
 - Se puede levantar / bajar el lápiz (up, down)

Más Información:

<https://docs.python.org/2/library/turtle.html>

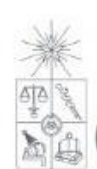


Turtle

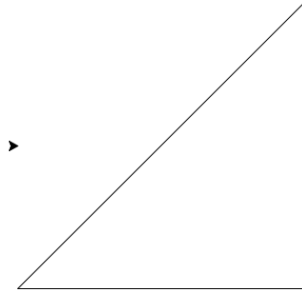
- Puede rotar:
 - `turtle.left(angulo)`
 - `turtle.right(angulo)`
 - angulo en grados, pero se puede cambiar:
 - `turtle.degrees()`, `turtle.radians()`
- Puede ser llevada un punto (x, y):
 - `turtle.setposition(x, y)`
- Puede indicar dónde está
 - `x = turtle.xcor()`
 - `y = turtle.ycor()`

Más Información:

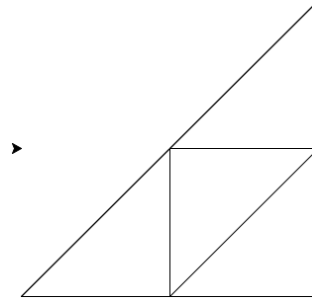
<https://docs.python.org/2/library/turtle.html>



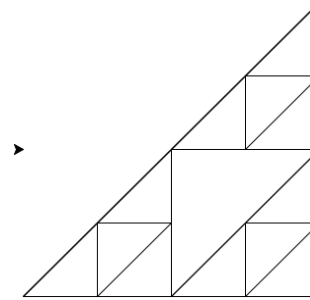
Triángulo de Sierpinski



$n = 1$

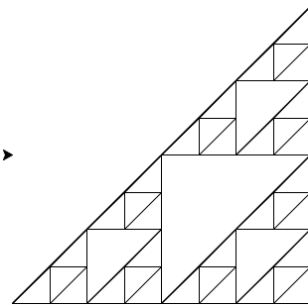


$n = 2$

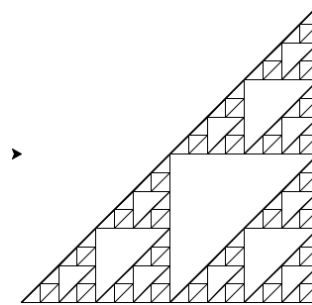


$n = 3$

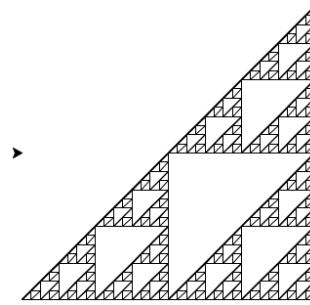
`dibujarSierpinski(0, 0, 300, 0, 300, 300, n)`



$n = 4$



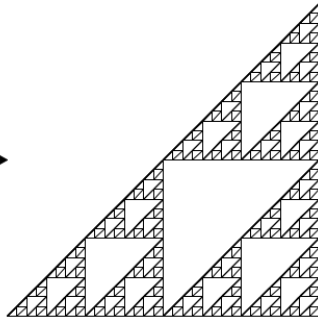
$n = 5$



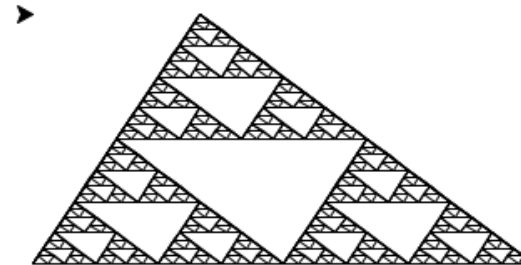
$n = 6$

Triángulo de Sierpinski

- Dibuja recursivamente el triángulo de Sierpinski en profundidad n:
-
- `# dibujarSierpinski: num num num num num num int -->`
- `# (xi, yi) son lo vertices del triangulo`
- `# n es la profundidad, con n >= 1`
- `def dibujarSierpinski(x1, y1, x2, y2, x3, y3, n):`

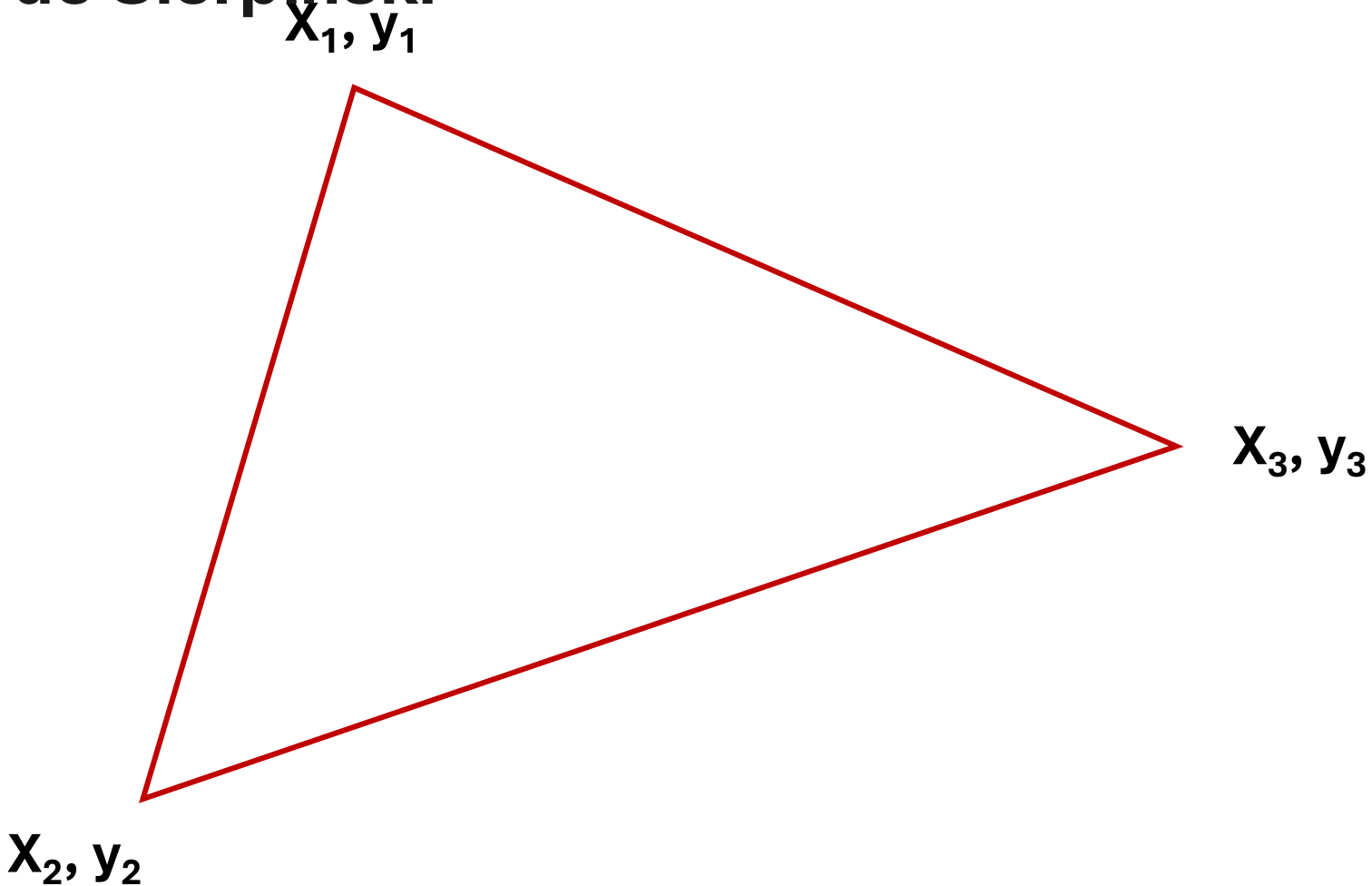


`dibujarSierpinski(0, 0, 300, 0, 300, 300, 6)`

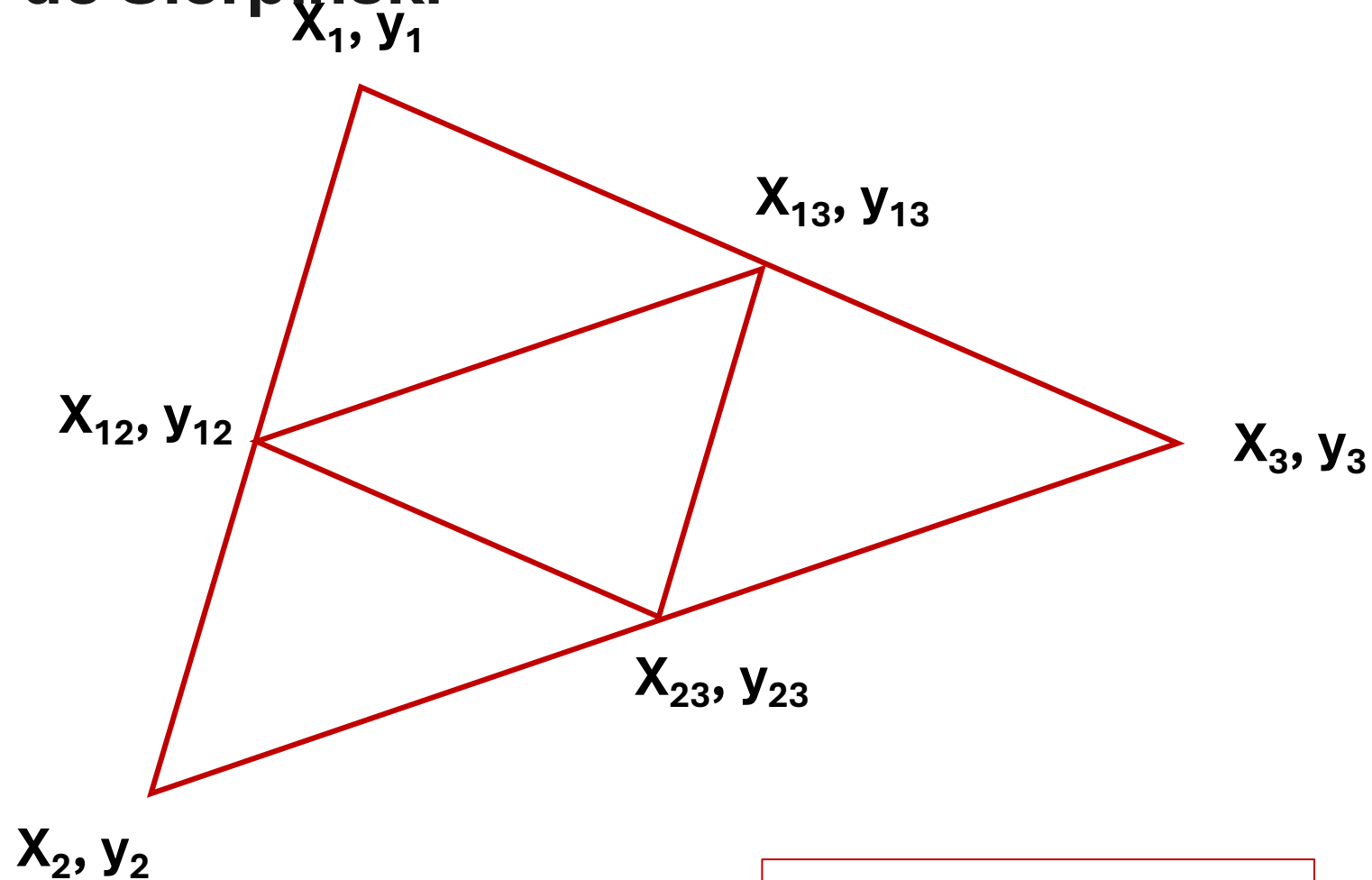


`dibujarSierpinski(0, 0, 300, 0, 100, 150, 6)`

Triángulo de Sierpinski



Triángulo de Sierpinski

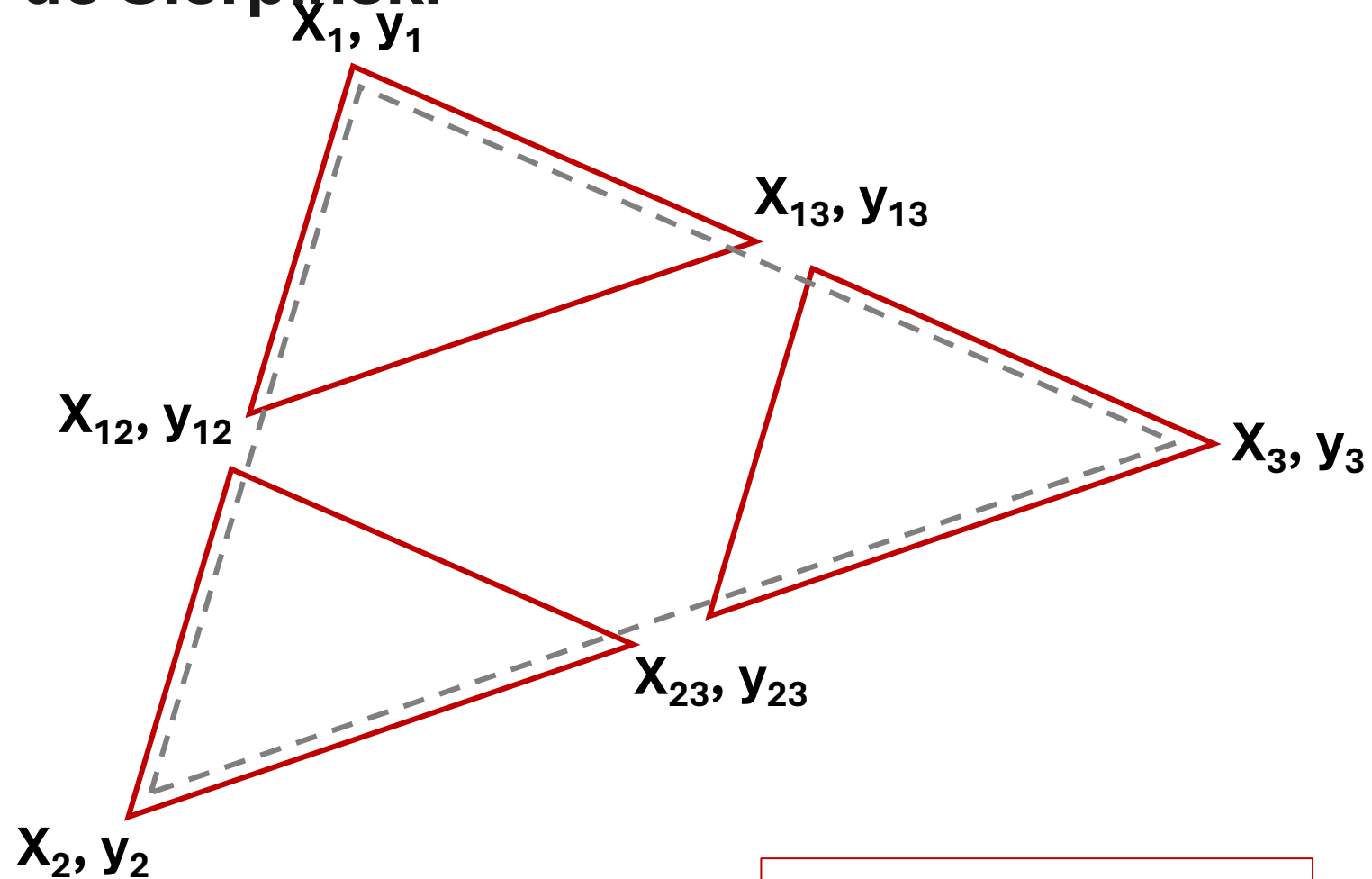


Donde:

$$X_{ab} = (X_a + X_b) / 2$$

$$y_{ab} = (y_a + y_b) / 2$$

Triángulo de Sierpinski



Donde:

$$X_{ab} = (X_a + X_b) / 2$$

$$y_{ab} = (y_a + y_b) / 2$$

Dibujando una línea

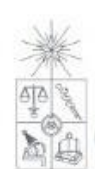
```
import turtle

# dibujarLinea: int int int int ->
# dados los puntos (x1, y1) y (x2, y2), dibuja
# una línea recta que une ambos puntos
# Ejemplo: dibujarLinea(0, 0, 200, 200)
def dibujarLinea(x1, y1, x2, y2):
    turtle.penup()           # levanta el lápiz
    turtle.setposition(x1, y1) # ubica el lápiz
    turtle.down()            # baja el lápiz
    turtle.setposition(x2, y2) # dibuja la línea

dibujarLinea(0, 0, 200, 200)
```

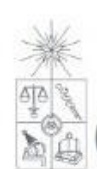
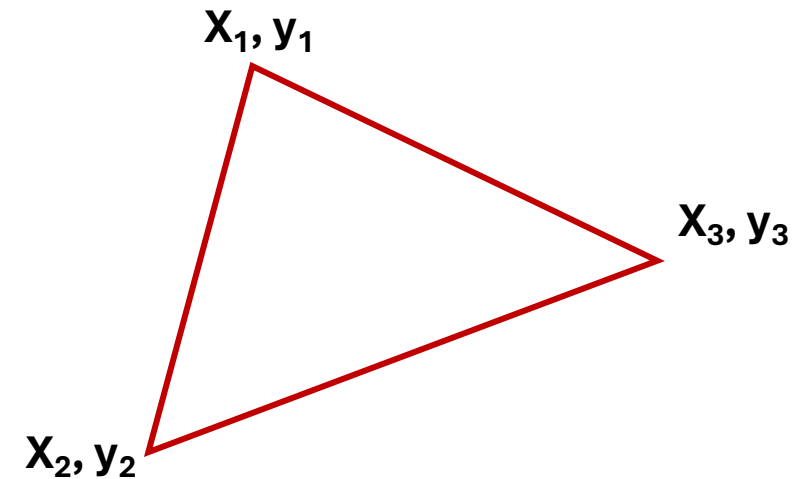
Ojo

El origen (0, 0) queda en el centro de la pantalla, no en una esquina.



Dibujando un triángulo

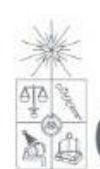
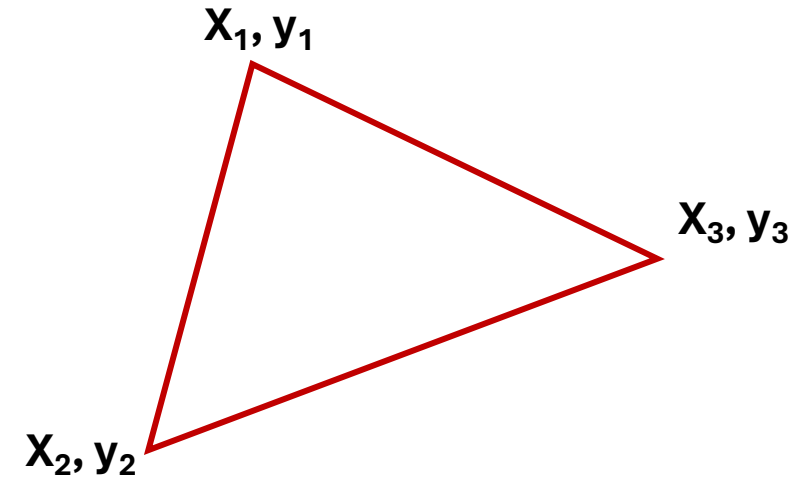
```
# dibujarTriangulo: int int int int int int -->  
# dada las coordenadas de tres puntos,  
# dibuja un triangulo  
def dibujarTriangulo(x1, y1, x2, y2, x3, y3):  
    dibujarLinea(x1, y1, x2, y2)  
    dibujarLinea(x2, y2, x3, y3)  
    dibujarLinea(x3, y3, x1, y1)
```



Dibujando un triángulo

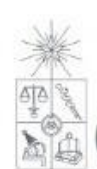
```
# dibujarTriangulo: int int int int int int -->  
# dada las coordenadas de tres puntos,  
# dibuja un triangulo  
def dibujarTriangulo(x1, y1, x2, y2, x3, y3):  
    dibujarLinea(x1, y1, x2, y2)  
    dibujarLinea(x2, y2, x3, y3)  
    dibujarLinea(x3, y3, x1, y1)
```

```
def dibujarLinea(x1, y1, x2, y2):  
    turtle.penup()  
    turtle.setposition(x1, y1)  
    turtle.down()  
    turtle.setposition(x2, y2)
```



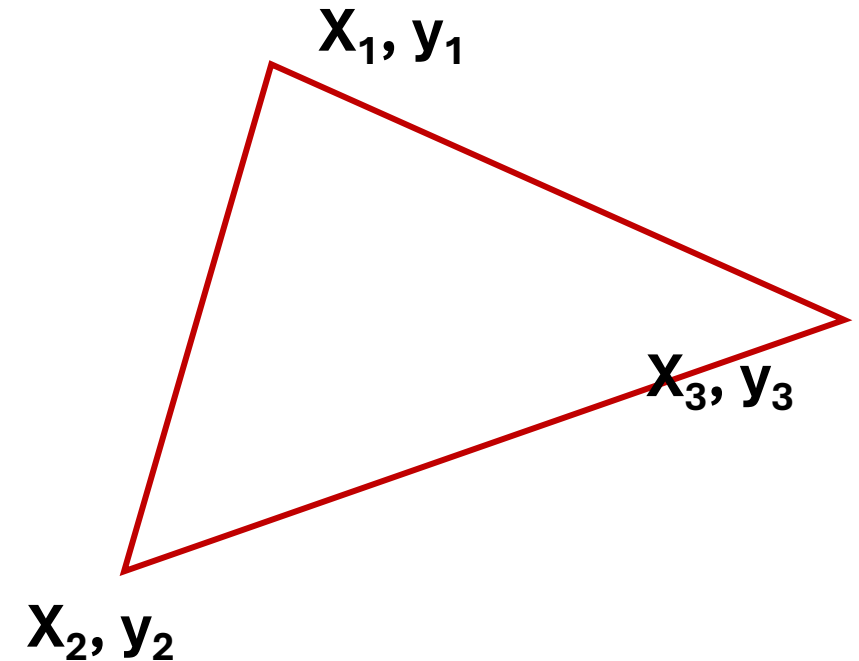
Dado dos valores, retorna el valor medio

```
# medio: int int --> float
# dado dos valores, retorna el valor medio
# Ejemplo: medio(1, 2) retorna 1.5
def medio(a, b):
    return (a + b)/2.0
# test
assert cerca(medio(1, 2), 1.5, 0.0001)
```



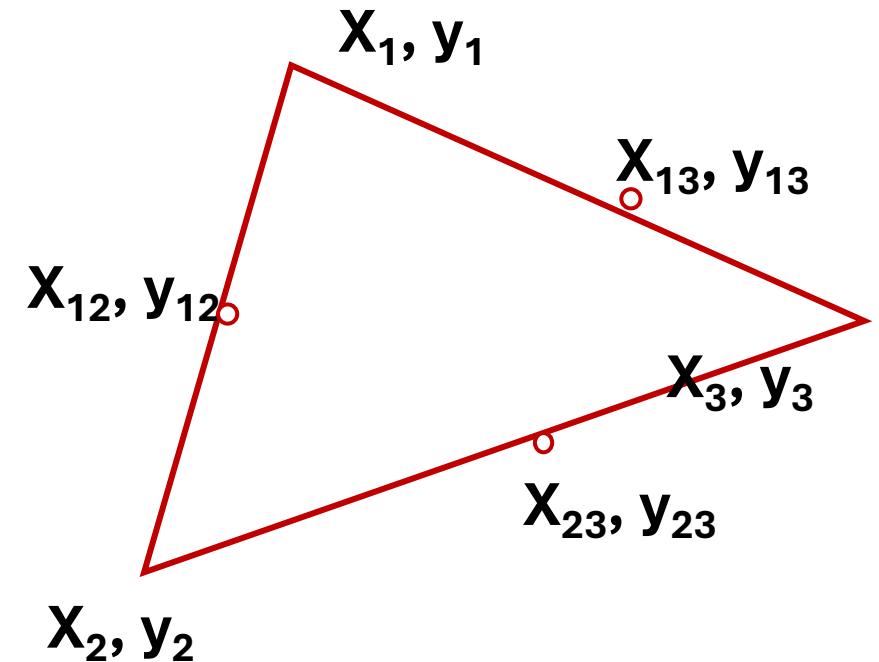
Triángulo de Sierpinsky

```
# dibujarSierpinsky: int int int int int int int --> None
# dado un triangulo, lo dibuja y luego crea recursivamente
# tres nuevos triangulos formado por los puntos medios de
# cada lado del triangulo
def dibujarSierpinsky(x1, y1, x2, y2, x3, y3, n):
    if n >= 1:
        dibujarTriangulo(x1, y1, x2, y2, x3, y3)
        x12 = medio(x1, x2)
        y12 = medio(y1, y2)
        x23 = medio(x2, x3)
        y23 = medio(y2, y3)
        x13 = medio(x1, x3)
        y13 = medio(y1, y3)
        dibujarSierpinsky( x1, y1, x12, y12, x13, y13, n-1)
        dibujarSierpinsky(x12, y12, x2, y2, x23, y23, n-1)
        dibujarSierpinsky(x23, y23, x3, y3, x13, y13, n-1)
```



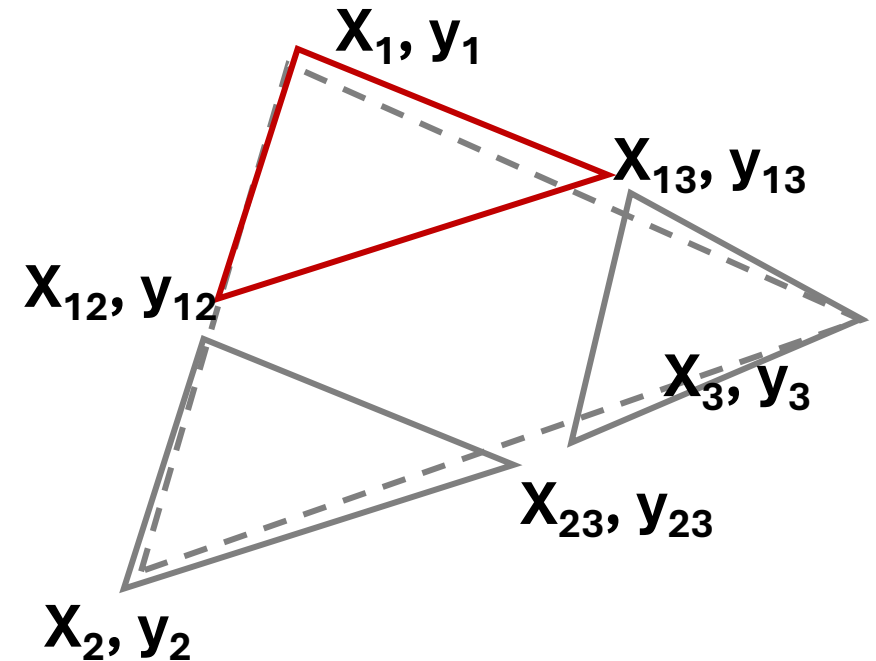
Triángulo de Sierpinsky

```
# dibujarSierpinsky: int int int int int int int --> None
# dado un triangulo, lo dibuja y luego crea recursivamente
# tres nuevos triangulos formado por los puntos medios de
# cada lado del triangulo
def dibujarSierpinsky(x1, y1, x2, y2, x3, y3, n):
    if n >= 1:
        dibujarTriangulo(x1, y1, x2, y2, x3, y3)
        x12 = medio(x1, x2)
        y12 = medio(y1, y2)
        x23 = medio(x2, x3)
        y23 = medio(y2, y3)
        x13 = medio(x1, x3)
        y13 = medio(y1, y3)
        dibujarSierpinsky( x1,  y1, x12, y12, x13, y13, n-1)
        dibujarSierpinsky(x12, y12,  x2,  y2, x23, y23, n-1)
        dibujarSierpinsky(x23, y23,  x3,  y3, x13, y13, n-1)
```



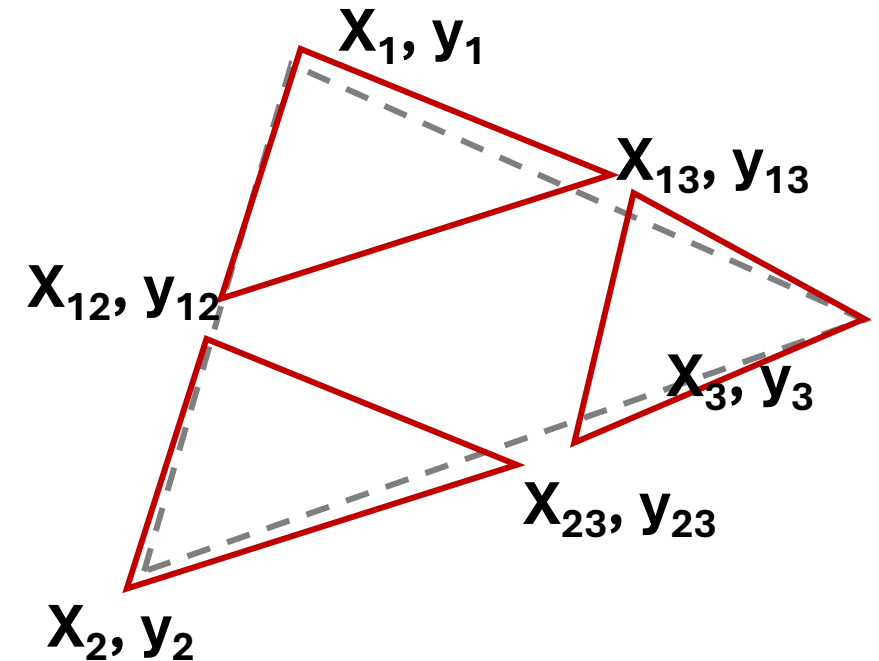
Triángulo de Sierpinsky

```
# dibujarSierpinsky: int int int int int int int --> None
# dado un triangulo, lo dibuja y luego crea recursivamente
# tres nuevos triangulos formado por los puntos medios de
# cada lado del triangulo
def dibujarSierpinsky(x1, y1, x2, y2, x3, y3, n):
    if n >= 1:
        dibujarTriangulo(x1, y1, x2, y2, x3, y3)
        x12 = medio(x1, x2)
        y12 = medio(y1, y2)
        x23 = medio(x2, x3)
        y23 = medio(y2, y3)
        x13 = medio(x1, x3)
        y13 = medio(y1, y3)
        dibujarSierpinsky(x1, y1, x12, y12, x13, y13, n-1)
        dibujarSierpinsky(x12, y12, x2, y2, x23, y23, n-1)
        dibujarSierpinsky(x23, y23, x3, y3, x13, y13, n-1)
```



Triángulo de Sierpinsky

```
# dibujarSierpinsky: int int int int int int int --> None
# dado un triangulo, lo dibuja y luego crea recursivamente
# tres nuevos triangulos formado por los puntos medios de
# cada lado del triangulo
def dibujarSierpinsky(x1, y1, x2, y2, x3, y3, n):
    if n >= 1:
        dibujarTriangulo(x1, y1, x2, y2, x3, y3)
        x12 = medio(x1, x2)
        y12 = medio(y1, y2)
        x23 = medio(x2, x3)
        y23 = medio(y2, y3)
        x13 = medio(x1, x3)
        y13 = medio(y1, y3)
        dibujarSierpinsky(x1, y1, x12, y12, x13, y13, n-1)
        dibujarSierpinsky(x12, y12, x2, y2, x23, y23, n-1)
        dibujarSierpinsky(x23, y23, x3, y3, x13, y13, n-1)
```



Propuesto

- Dado un número, escriba en pantalla el dígito que este en su posición, numerados de derecha a izquierda.

- Ejemplo:

`escribirEnPosicion(74123)`

2

4



`escribirEnPosicion(654103)`

4

5

6

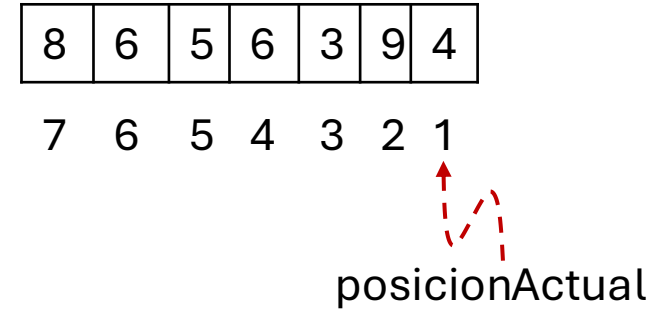


En este caso SI se puede usar
print en la función



Solución Propuesto

- El primer dígito es "simple"
 $\text{digito} = \text{numero} \% 10$
 $\text{posicionActual} = 1$
 if $\text{digito} == \text{posicionActual}$:
 print num



Solución Propuesto

- El primer dígito es "simple"

`digito = numero % 10`

`posicionActual = 1`

`if digito == posicionActual :`

`print num`

8	6	5	6	3	9	4
---	---	---	---	---	---	---

7 6 5 4 3 2 1


posicionActual

- Para el resto hay que recuperar la posición y el siguiente dígito

`::`

`else:`

`::`

`if digito == posicionActual:`

`print num`

8	6	5	6	3	9
---	---	---	---	---	---

7 6 5 4 3 2


posicionActual

Solución Propuesto

- El primer dígito es "simple"

$\text{digito} = \text{numero} \% 10$

$\text{posicionActual} = 1$

 if $\text{digito} == \text{posicionActual}$:

 print num

- Para el resto hay que recuperar la posición y el siguiente dígito

 ::

 else:

 ::

 if $\text{digito} == \text{posicionActual}$:

 print num

8	6	5	6	3	9	4
---	---	---	---	---	---	---

7 6 5 4 3 2 1

posicionActual

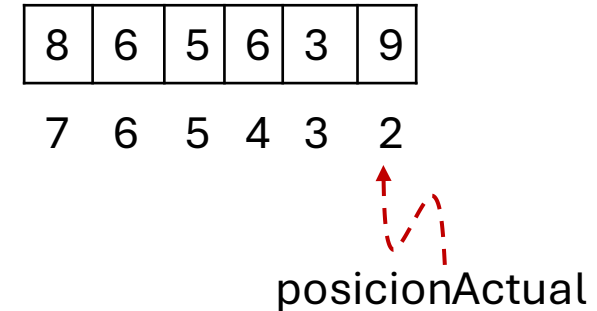
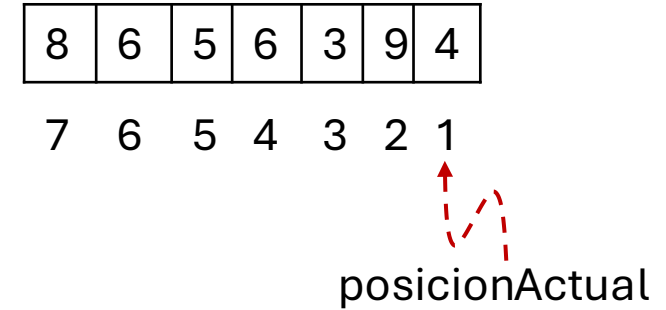
8	6	5	6	3	9
---	---	---	---	---	---

7 6 5 4 3 2

posicionActual

Solución Propuesto

- Dos variables que cambian
- `digito = numero % 10`
- `posicionActual = posicionActual + 1`
-
- Pero la función sólo tiene 1 parámetro:
- **def** escribirEnPosicion(numero):
- : :
-
- Solución:
- Crear una función auxiliar con más parámetros



```
def escribirEnPosicion(numero):  
    :  
    recorrerNumero(numero, posicionActual)
```

Función con un parámetro

Llama a

```
def recorrerNumero(num, posActual):  
    if num < 10:  
        if (num == posActual):  
            print(num)  
            return  
    else:  
        digito = num % 10  
        if digito == posActual:  
            print(digito)  
  
    recorrerNumero(num // 10, posActual + 1)
```

Función con dos parámetros

Función auxiliar: recorrerNumero

```
def escribirEnPosicion(numero):  
    posicionActual = 1  
    recorrerNumero(numero, posicionActual)
```

```
def recorrerNumero(num, posActual):  
    if num < 10:  
        if (num == posActual):  
            print(num)  
            return  
    else:  
        digito = num % 10  
        if digito == posActual:  
            print(digito)  
  
    recorrerNumero(num // 10, posActual + 1)
```

8	6	5	6	3	9	4
---	---	---	---	---	---	---

7 6 5 4 3 2 1



posicionActual

Función auxiliar: recorrerNumero

```
def escribirEnPosicion(numero):  
    posicionActual = 1  
    recorrerNumero(numero, posicionActual)
```

```
def recorrerNumero(num, posActual):  
    if num < 10:  
        if (num == posActual):  
            print(num)  
            return  
    else:  
        digito = num % 10  
        if digito == posActual:  
            print(digito)  
  
    recorrerNumero(num // 10, posActual + 1)
```

8	6	5	6	3	9	4
---	---	---	---	---	---	---

7 6 5 4 3 2 1



posicionActual

Función auxiliar: recorrerNumero

```
def escribirEnPosicion(numero):  
    posicionActual = 1  
    recorrerNumero(numero, posicionActual)
```

```
def recorrerNumero(num, posActual):  
    if num < 10:  
        if (num == posActual):  
            print(num)  
            return  
    else:  
        digito = num % 10  
        if digito == posActual:  
            print(digito)  
  
    recorrerNumero(num // 10, posActual + 1)
```

8	6	5	6	3	9	4
---	---	---	---	---	---	---

7 6 5 4 3 2 1



posicionActual

8	6	5	6	3	9
---	---	---	---	---	---

7 6 5 4 3 2



posicionActual

Función auxiliar: recorrerNumero

```
def escribirEnPosicion(numero):  
    posicionActual = 1  
    recorrerNumero(numero, posicionActual)
```

```
def recorrerNumero(num, posActual):  
    if num < 10:  
        if (num == posActual):  
            print(num)  
            return  
    else:  
        digito = num % 10  
        if digito == posActual:  
            print(digito)  
  
    recorrerNumero(num // 10, posActual + 1)
```

8	6	5	6	3	9	4
---	---	---	---	---	---	---

7 6 5 4 3 2 1



posicionActual

8	6	5	6	3	9
---	---	---	---	---	---

7 6 5 4 3 2



posicionActual

Función auxiliar: recorrerNumero

```
def escribirEnPosicion(numero):  
    posicionActual = 1  
    recorrerNumero(numero, posicionActual)
```

```
def recorrerNumero(num, posActual):  
    if num < 10:  
        if (num == posActual):  
            print(num)  
            return  
    else:  
        digito = num % 10  
        if digito == posActual:  
            print(digito)  
            recorrerNumero(num // 10, posActual + 1)
```

8	6	5	6	3	9	4
---	---	---	---	---	---	---

7 6 5 4 3 2 1

posicionActual

8	6	5	6	3	9
---	---	---	---	---	---

7 6 5 4 3 2

posicionActual